Project Title: Retail Sales Analytics Using Excel, SQL, Power BI and Python

1.1 Introduction

The **"Retail Sales Analytics Using Excel, SQL, Power BI and Python"** project is an end-to-end data analytics project focused on analyzing retail sales, customer behavior, product performance, inventory, stores, and staff performance. The project uses Excel for data cleaning and preparation, SQL for database management and querying, Python for exploratory data analysis and customer segmentation, and Power BI for interactive visualization and business reporting. The main objective is to transform raw retail data into meaningful business insights that can support better decision-making and improve sales, customer management, product performance, and inventory monitoring.

1.2 Background of the Project

The project focuses on analyzing retail data generated from customer transactions, orders, products, stores, staff, and inventory. A structured analytics process is required to convert raw transactional data into useful information. This project applies different analytics tools at different stages to clean, store, analyze, segment, and visualize the retail data.

1.3 Problem Statement

Retail businesses need to understand their sales performance, customer purchasing behavior, product performance, inventory levels, and staff performance. This project aims to develop an analytics solution that analyzes these areas and generates meaningful insights for business decision-making.

1.4 Objectives

The main objective of this project is to clean and prepare retail data using Excel, manage and analyze the data using SQL, perform exploratory data analysis and customer segmentation using Python, and develop an interactive Power BI dashboard to present important business insights.

1.5 Scope of the Project

The project covers the complete data analytics process from raw retail data preparation to business intelligence reporting. It includes data cleaning, database management, sales analysis, customer analysis, product analysis, inventory monitoring, RFM-based customer segmentation, staff performance analysis, and interactive Power BI reporting.

1.6 Business Use Cases

The project is designed to analyze sales performance across stores and staff, identify high-value customers and purchasing behavior, perform customer segmentation, monitor inventory and product performance, and provide management with an interactive dashboard for business insights.

1.7 Project Workflow

The project follows an end-to-end workflow starting with Excel for data cleaning and preparation, followed by SQL for database storage and querying, Python for exploratory analysis and customer segmentation, and finally Power BI for visualization, dashboarding, and business reporting

3 – EXCEL: DATA CLEANING AND PREPARATION

3.1 Standardizing Column Headers

The column headers across the different Excel worksheets were standardized to maintain consistent naming conventions and improve data readability and compatibility with SQL, Python, and Power BI.

3.2 Removing Duplicate Records

The datasets were checked for duplicate records and duplicate entries were removed where required to maintain data accuracy and prevent repeated records from affecting the analysis.

3.3 Handling Missing Values

Missing values were identified using Excel filtering and data inspection techniques. Relevant blank fields such as ZIP codes, phone numbers, and other missing values were reviewed and handled appropriately.

3.4 Data Type Conversion

The data types of different columns were checked and standardized. Date fields were converted into proper date formats, while numerical fields such as quantity, price, discount, and sales values were maintained in numeric format.

3.5 Data Validation

Data Validation was applied to selected fields to maintain consistency and prevent invalid entries. A dropdown list was created for the order_status field according to the status values available in the dataset.

3.6 Creating Derived Columns

A new total_price column was created in the order_items table to calculate the value of each order item after applying the discount. The calculation was based on list price multiplied by quantity minus discount.

3.7 Merging Lookup Data

VLOOKUP was used to merge relevant product information into the order_items worksheet using product_id as the matching key. This helped combine information from related worksheets for further analysis.

3.8 Pivot Table Analysis

A basic Pivot Table was created to summarize total sales by product category. This provided a simple comparison of sales performance across different product categories.

3.9 Outlier Analysis

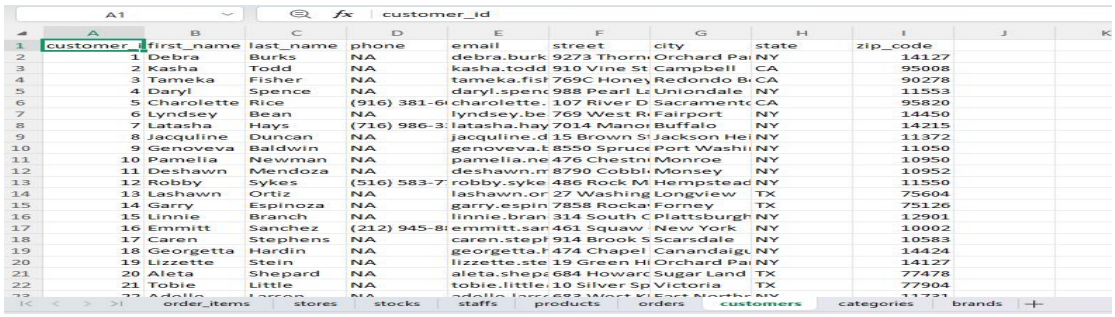
Product prices were sorted and filtered to identify unusually high or low list_price values. This helped review potential pricing outliers before further analysis.

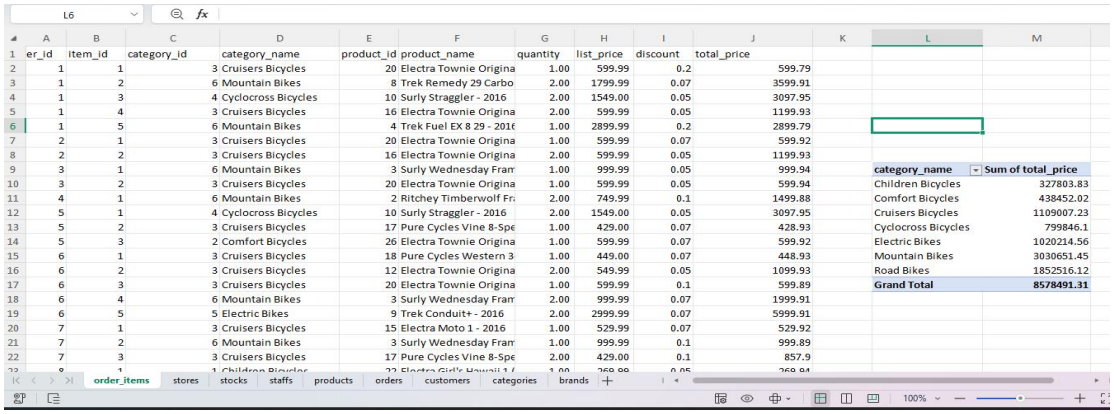
3.10 Preparing Final CSV Files

After completing the cleaning and preparation activities, the cleaned worksheets were prepared as CSV files for importing into the SQL database. These activities follow the Excel requirements provided in the project task sheet.

3.11 Excel Analysis Summary

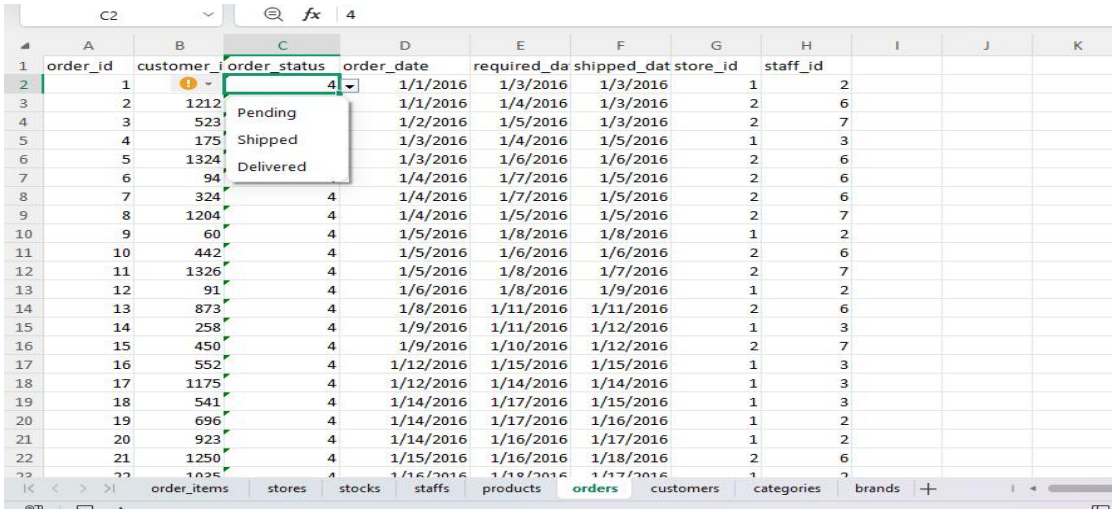
The Excel stage focused on cleaning, validating, transforming, and preparing the retail datasets for further analysis. Column headers and data types were standardized, duplicate and missing records were reviewed, Data Validation was applied, and derived values such as total_price were created. VLOOKUP was used to combine relevant product information, while Pivot Table analysis and price outlier checking were performed to understand the data. Finally, the cleaned worksheets were prepared as CSV files and used for the next stage of the project, SQL database management and querying.





er_id	item_id	category_id	category_name	product_id	product_name	quantity	list_price	discount	total_price
1	1	3	Cruisers Bicycles	20	Electra Townie Origina	1.00	599.99	0.2	599.79
1	2	6	Mountain Bikes	8	Trek Remedy 29 Carbo	2.00	1799.99	0.07	3599.91
1	3	4	Cyclocross Bicycles	10	Surly Straggler - 2016	2.00	1549.00	0.05	3097.95
1	4	3	Cruisers Bicycles	16	Electra Townie Origina	2.00	599.99	0.05	1199.93
1	5	6	Mountain Bikes	4	Trek Fuel EX 8 29 - 2016	1.00	2899.99	0.2	2899.79
2	1	3	Cruisers Bicycles	20	Electra Townie Origina	1.00	599.99	0.07	599.92
2	2	3	Cruisers Bicycles	16	Electra Townie Origina	2.00	599.99	0.05	1199.93
3	1	6	Mountain Bikes	3	Surly Wednesday Fram	1.00	999.99	0.05	999.94
3	2	3	Cruisers Bicycles	20	Electra Townie Origina	1.00	599.99	0.05	599.94
4	1	6	Mountain Bikes	2	Ritchey Timberwolf Fr	2.00	749.99	0.1	1499.88
5	1	4	Cyclocross Bicycles	10	Surly Straggler - 2016	2.00	1549.00	0.05	3097.95
5	2	3	Cruisers Bicycles	17	Pure Cycles Vine 8-Spe	1.00	429.00	0.07	428.93
5	3	2	Comfort Bicycles	26	Electra Townie Origina	1.00	599.99	0.07	599.92
6	1	3	Cruisers Bicycles	18	Pure Cycles Western 3	1.00	449.00	0.07	448.93
6	2	3	Cruisers Bicycles	12	Electra Townie Origina	2.00	549.99	0.05	1099.93
6	3	3	Cruisers Bicycles	20	Electra Townie Origina	1.00	599.99	0.1	599.89
6	4	6	Mountain Bikes	3	Surly Wednesday Fram	2.00	999.99	0.07	1999.91
6	5	5	Electric Bikes	9	Trek Conduit+ - 2016	2.00	2999.99	0.07	5999.91
7	1	3	Cruisers Bicycles	15	Electra Moto 1 - 2016	1.00	529.99	0.07	529.92
7	2	6	Mountain Bikes	3	Surly Wednesday Fram	1.00	999.99	0.1	999.89
7	3	3	Cruisers Bicycles	17	Pure Cycles Vine 8-Spe	2.00	429.00	0.1	857.9

category_name	Sum of total_price
Children Bicycles	327803.83
Comfort Bicycles	438452.02
Cruisers Bicycles	1109007.23
Cyclocross Bicycles	799846.1
Electric Bikes	1020214.56
Mountain Bikes	3030651.45
Road Bikes	1852516.12
Grand Total	**8578491.31**



order_id	customer_i	order_status	order_date	required_da	shipped_dat	store_id	staff_id
1		4	1/1/2016	1/3/2016	1/3/2016	1	2
2	1212		1/1/2016	1/4/2016	1/3/2016	2	6
3	523		1/2/2016	1/5/2016	1/3/2016	2	7
4	175		1/3/2016	1/4/2016	1/5/2016	1	3
5	1324		1/3/2016	1/6/2016	1/6/2016	2	6
6	94		1/4/2016	1/7/2016	1/5/2016	2	6
7	324	4	1/4/2016	1/7/2016	1/5/2016	2	6
8	1204	4	1/4/2016	1/5/2016	1/5/2016	2	7
9	60	4	1/5/2016	1/8/2016	1/8/2016	1	2
10	442	4	1/5/2016	1/6/2016	1/6/2016	2	6
11	1326	4	1/5/2016	1/8/2016	1/7/2016	2	7
12	91	4	1/6/2016	1/8/2016	1/9/2016	1	2
13	873	4	1/8/2016	1/11/2016	1/11/2016	2	6
14	258	4	1/9/2016	1/11/2016	1/12/2016	1	3
15	450	4	1/9/2016	1/10/2016	1/12/2016	2	7
16	552	4	1/12/2016	1/15/2016	1/15/2016	1	3
17	1175	4	1/12/2016	1/14/2016	1/14/2016	1	3
18	541	4	1/14/2016	1/17/2016	1/15/2016	1	3
19	696	4	1/14/2016	1/17/2016	1/16/2016	1	2
20	923	4	1/14/2016	1/16/2016	1/17/2016	1	2
21	1250	4	1/15/2016	1/16/2016	1/18/2016	2	6

4 – SQL: DATABASE MANAGEMENT AND QUERYING

4.1 Creating Database and Tables

A relational database was created in MySQL to store and manage the cleaned retail sales data. The database was structured according to the provided Entity Relationship Diagram (ERD). Tables such as customers, orders, order_items, products, brands, categories, stocks, stores, and staffs were created with appropriate primary keys, foreign keys, and data types.

4.2 Defining Primary and Foreign Keys

Primary keys were defined for uniquely identifying records in each table. Foreign key relationships were established between related tables to maintain referential integrity. For example, customer_id in the orders table was linked to the customers table, while store_id and staff_id were linked to the stores and staffs tables respectively.

4.3 Importing CSV Data into SQL

The cleaned CSV files prepared during the Excel stage were imported into the MySQL database. The data was loaded into the corresponding tables using MySQL Workbench import functionality. The imported records were verified to ensure that the data was transferred correctly without affecting the table structure.

4.4 Inner Join for Order Details

An INNER JOIN was used to combine the orders, order_items, and products tables. This query was used to retrieve detailed order information, including order details, product names, quantities, prices, and total sales values. The join operation helped combine information stored across multiple related tables for detailed analysis.

4.5 Total Sales by Store

A SQL aggregation query was created to calculate the total sales generated by each store. The SUM() function was used on the total_price field and the results were grouped using store_id. This analysis helped compare sales performance across different stores.

4.6 Top 5 Selling Products

The top five selling products were identified based on the total quantity sold. The SUM() function was used to calculate the total quantity for each product, followed by GROUP BY to summarize the results. The results were sorted in descending order using ORDER BY and limited to the top five records using LIMIT 5.

4.7 Customer Purchase Summary

A customer purchase summary was created by combining customer and order information. The analysis calculated the total number of orders, total items purchased, and total revenue generated by each customer. This helped identify customer purchasing patterns and understand the contribution of individual customers to overall sales.

4.8 Customer Segmentation by Total Spend

Customers were classified into different spending categories based on their total purchase value. A CASE statement was used to create spending brackets such as Low, Medium, and High. This segmentation helped identify customers based on their purchasing value and provided useful information for targeted customer analysis.

4.9 Staff Performance Analysis

Staff performance was analyzed by calculating the total revenue generated from the orders handled by each staff member. The orders, order_items, and staffs tables were joined to connect staff members with their handled orders and corresponding sales. The results were grouped by staff member to compare revenue performance.

4.10 Stock Alert Analysis

A stock alert query was created to identify products with low inventory levels. Products with a stock quantity of less than 10 units were identified across different stores. This analysis helps monitor inventory levels and identify products that may require replenishment. The project task specifically defines the stock-alert threshold as below 10 units.

4.11 Creating Customer Segmentation Table

A separate customer_segments table was created in the SQL database to store customer segmentation results. This table is designed to receive the segmentation results generated during the Python and machine learning stage. The table can later be imported into Power BI for customer segment visualization and analysis.

4.12 SQL Data Validation

After completing the SQL operations, the tables and query results were reviewed to verify data accuracy and consistency. Primary key and foreign key relationships were checked, and aggregation results were reviewed to ensure that sales, customer, staff, and inventory analyses were generated correctly.

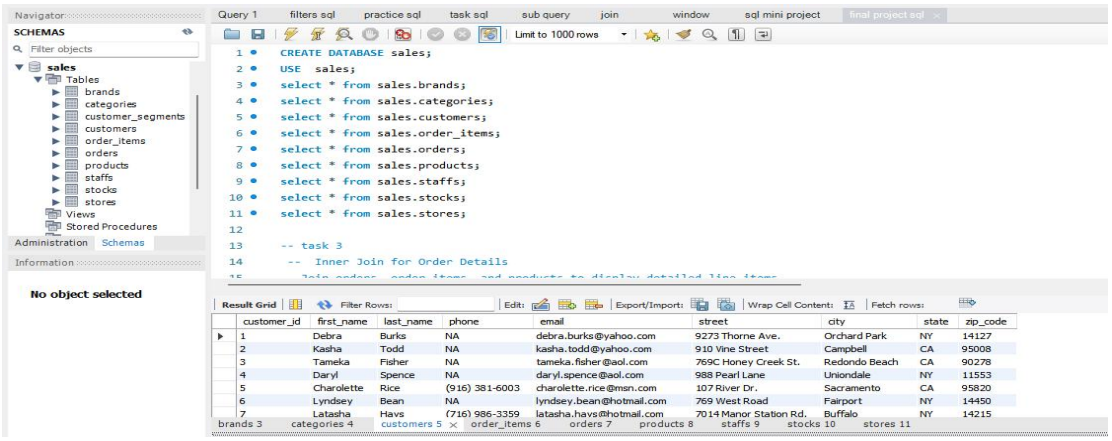
4.13 SQL Analysis Summary

The SQL stage transformed the cleaned Excel data into a structured relational database and enabled analytical queries across multiple retail entities. The SQL analysis covered order details, store sales, top-selling products, customer purchasing behavior, customer spending categories, staff performance, and low-stock products. These outputs provide the foundation for the subsequent Python analysis and Power BI dashboard development. The project specification identifies SQL as the stage for data storage, querying, and transformation.

4.14 Preparing Data for Python and Power BI

The analyzed SQL tables were prepared for the next stages of the project. The orders, order_items, and customers tables are used for Python-based exploratory analysis and RFM calculation, while key tables such as orders, products, order_items, customers, stores, and staffs are later connected to Power BI for visualization and reporting.

This structure matches your Excel chapter style and follows the actual SQL requirements in your project task sheet, so you can directly continue your report with – SQL.


Navigator:
SCHEMAS
Filter objects
sales
Tables
brands
categories
customer_segments
customers
order_items
orders
products
staffs
stocks
stores
Views
Stored Procedures
Administration
Schemas
Information
No object selected
Query 1
filters sql
practice sql
task sql
sub query
join
window
sql mini project
final project sql
Limit to 1000 rows
1 CREATE DATABASE sales;
2 USE sales;
3 select * from sales.brands;
4 select * from sales.categories;
5 select * from sales.customers;
6 select * from sales.order_items;
7 select * from sales.orders;
8 select * from sales.products;
9 select * from sales.staffs;
10 select * from sales.stocks;
11 select * from sales.stores;
13 -- task 3
14 -- Inner Join for Order Details
Result Grid
Filter Rows:
Edit:
Export/Import:
Wrap Cell Content:
Fetch rows:
customer_id | first_name | last_name | phone | email | street | city | state | zip_code
1 | Debra | Burks | NA | debra.burks@yahoo.com | 9273 Thorne Ave. | Orchard Park | NY | 14127
2 | Kasha | Todd | NA | kasha.todd@yahoo.com | 910 Vine Street | Campbell | CA | 95008
3 | Tameka | Fisher | NA | tameka.fisher@aol.com | 769C Honey Creek St. | Redondo Beach | CA | 90278
4 | Daryl | Spence | NA | daryl.spence@aol.com | 988 Pearl Lane | Uniondale | NY | 11553
5 | Charolette | Rice | (916) 381-6003 | charolette.rice@msn.com | 107 River Dr. | Sacramento | CA | 95820
6 | Lyndsey | Bean | NA | lyndsey.bean@hotmail.com | 769 West Road | Fairport | NY | 14450
7 | Latasha | Hays | (716) 986-3359 | latasha.hays@hotmail.com | 7014 Manor Station Rd. | Buffalo | NY | 14215
brands 3
categories 4
customers 5
order_items 6
orders 7
products 8
staffs 9
stocks 10
stores 11

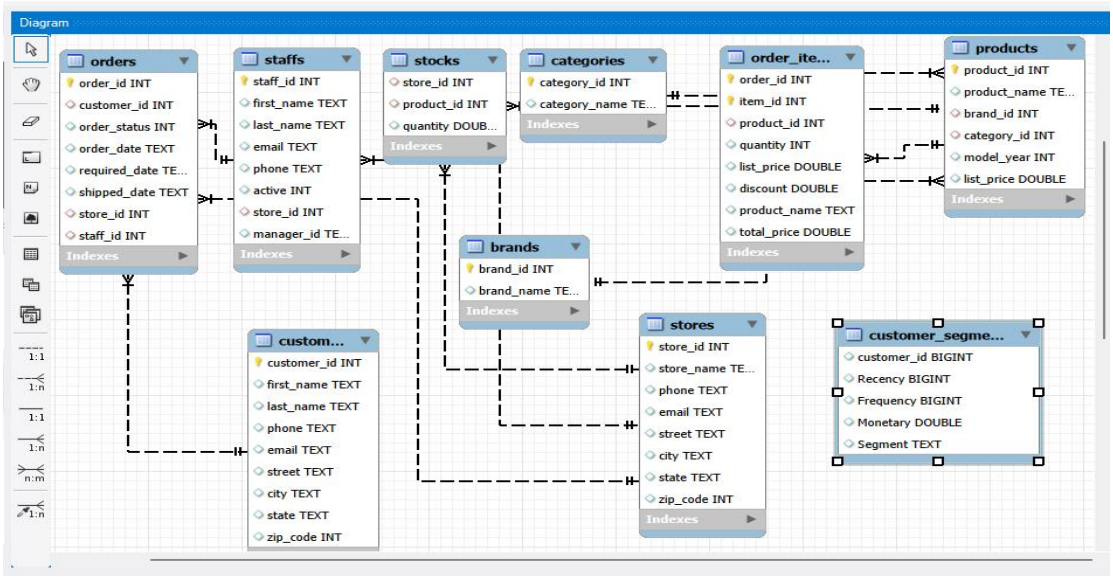
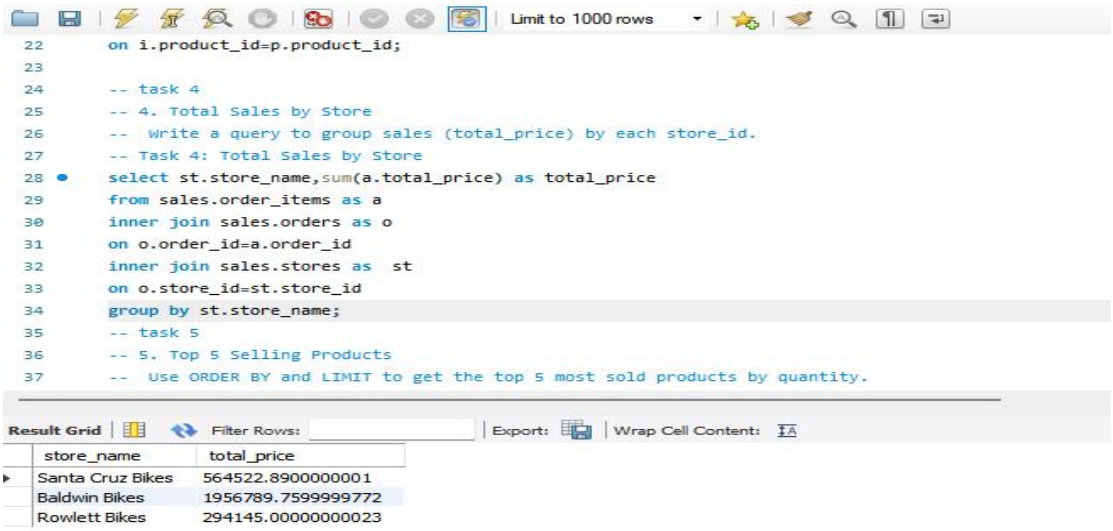


Diagram
orders
order_id INT
customer_id INT
order_status INT
order_date TEXT
required_date TE...
shipped_date TEXT
store_id INT
staff_id INT
Indexes
staffs
staff_id INT
first_name TEXT
last_name TEXT
email TEXT
phone TEXT
active INT
store_id INT
manager_id TE...
Indexes
stocks
store_id INT
product_id INT
quantity DOUB...
Indexes
categories
category_id INT
category_name TE...
Indexes
order_ite...
order_id INT
item_id INT
product_id INT
quantity INT
list_price DOUBLE
discount DOUBLE
product_name TEXT
total_price DOUBLE
Indexes
products
product_id INT
product_name TE...
brand_id INT
category_id INT
model_year INT
list_price DOUBLE
Indexes
brands
brand_id INT
brand_name TE...
Indexes
custom...
customer_id INT
first_name TEXT
last_name TEXT
phone TEXT
email TEXT
street TEXT
city TEXT
state TEXT
zip_code INT
stores
store_id INT
store_name TE...
phone TEXT
email TEXT
street TEXT
city TEXT
state TEXT
zip_code INT
Indexes
customer_segme...
customer_id BIGINT
Recency BIGINT
Frequency BIGINT
Monetary DOUBLE
Segment TEXT
1:1
1:n
1:1
1:n
n:m
1:n


Limit to 1000 rows
on i.product_id=p.product_id;
-- task 4
-- 4. Total Sales by Store
-- Write a query to group sales (total_price) by each store_id.
-- Task 4: Total Sales by Store
select st.store_name,sum(a.total_price) as total_price
from sales.order_items as a
inner join sales.orders as o
on o.order_id=a.order_id
inner join sales.stores as st
on o.store_id=st.store_id
group by st.store_name;
-- task 5
-- 5. Top 5 Selling Products
-- Use ORDER BY and LIMIT to get the top 5 most sold products by quantity.
Result Grid
Filter Rows:
Export:
Wrap Cell Content:
store_name | total_price
Santa Cruz Bikes | 564522.8900000001
Baldwin Bikes | 1956789.7599999772
Rowlett Bikes | 294145.00000000023

```sql
select p.product_id,p.product_name,sum(o.quantity) as total_quantity
from sales.order_items as o
inner join sales.products as p
on o.product_id = p.product_id
group by p.product_id, p.product_name
order by total_quantity desc
limit 5;
-- 6. Customer Purchase Summary
-- For each customer, return total orders placed, total items purchased, and total revenue.
select c.customer_id,c.first_name,count(o.order_id)as total_orders,sum(i.quantity)as total_items,sum(i.total_price) as total_revenue
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

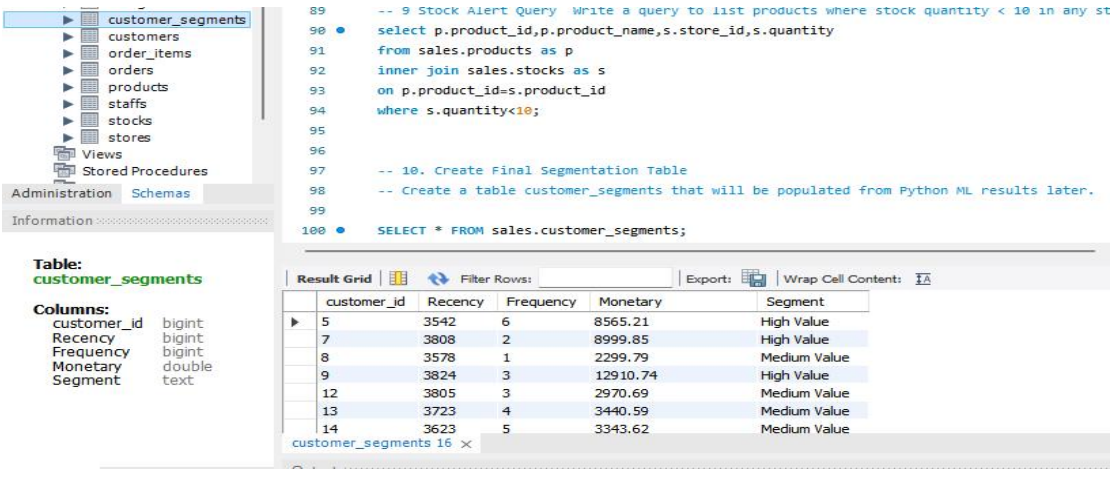
product_id	product_name	total_quantity
13	Electra Cruiser 1 (24-Inch) - 2016	115
7	Trek Slash 8 27.5 - 2016	111
6	Surly Ice Cream Truck Frameset - 2016	108
25	Electra Townie Original 7D - 2015/2016	108
12	Electra Townie Original 21D - 2016	107

```sql
-- 9 Stock Alert Query  Write a query to list products where stock quantity < 10 in any store.
select p.product_id,p.product_name,s.store_id,s.quantity
from sales.products as p
inner join sales.stocks as s
on p.product_id=s.product_id
where s.quantity<10;


-- 10. Create Final Segmentation Table
-- Create a table customer_segments that will be populated from Python ML results later.

SELECT * FROM sales.customer_segments;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	store_id	quantity
2	Ritchey Timberwolf Frameset - 2016	1	5
3	Surly Wednesday Frameset - 2016	1	6
6	Surly Ice Cream Truck Frameset - 2016	1	0
7	Trek Slash 8 27.5 - 2016	1	8
8	Trek Remedy 29 Carbon Frameset - 2016	1	0
11	Surly Straggler 650b - 2016	1	8
14	Electra Girl's Hawaii 1 (16-inch) - 2015/2016	1	8

esult 15




```sql
-- 9 Stock Alert Query  Write a query to list products where stock quantity < 10 in any st
select p.product_id,p.product_name,s.store_id,s.quantity
from sales.products as p
inner join sales.stocks as s
on p.product_id=s.product_id
where s.quantity<10;


-- 10. Create Final Segmentation Table
-- Create a table customer_segments that will be populated from Python ML results later.

SELECT * FROM sales.customer_segments;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	Recency	Frequency	Monetary	Segment
5	3542	6	8565.21	High Value
7	3808	2	8999.85	High Value
8	3578	1	2299.79	Medium Value
9	3824	3	12910.74	High Value
12	3805	3	2970.69	Medium Value
13	3723	4	3440.59	Medium Value
14	3623	5	3343.62	Medium Value

customer_segments 16

5 PYTHON: EXPLORATORY DATA ANALYSIS AND CUSTOMER SEGMENTATION

5.1 Connecting Python with SQL

Python was connected to the MySQL database using SQLAlchemy and PyMySQL. The database connection was established using a SQLAlchemy engine, which enabled the Python environment to retrieve data directly from the SQL database. The orders, order_items, and customers tables were loaded into Pandas DataFrames using pandas.read_sql().

5.2 Loading Data from SQL

The retail data stored in MySQL was imported into Python for further analysis. The orders, order_items, and customers tables were separately loaded into Pandas DataFrames. The imported datasets contained 1,615 order records, 1,615 order-item records, and 1,445 customer records respectively.

5.3 Exploratory Data Analysis

Basic Exploratory Data Analysis (EDA) was performed using Pandas functions such as info(), describe(), count(), and value_counts(). These functions were used to understand the structure, data types, record counts, numerical distributions, and categorical values in the datasets.

5.4 Statistical Summary of Order Items

The describe() function was used to obtain statistical summaries of the order-item data. The analysis included quantity, list price, discount, and total price. The dataset contained 1,615 records, with an average total price of approximately 1,743.32 and a maximum total price of approximately 23,999.88.

5.5 Order Status Analysis

The order dataset was analyzed using value_counts() to understand the distribution of order statuses. The analysis showed 1,445 records with order status 4, 63 with status 2, 62 with status 1, and 45 with status 3. This helped understand the distribution of orders across the available status categories.

5.6 Product Frequency Analysis

The frequency of products appearing in the order-item dataset was analyzed using value_counts(). Product IDs were counted to identify how frequently each product appeared in the transaction data. Product IDs 12 and 13 appeared most frequently with 72 records each, followed by product ID 6 with 72 records.

5.7 Customer Distribution Analysis

Customer data was analyzed using Pandas to understand the distribution of customers across different states. The analysis identified customers from New York (NY), California (CA), and Texas (TX), with NY having the highest number of customer records.

5.8 Monthly Orders Trend

A monthly order trend was created using Matplotlib. The order date was converted into a proper datetime format and orders were grouped by month. A line chart was then created to visualize the number of orders across the months. This helped identify variations in monthly order activity.

5.9 List Price Distribution Analysis

A distribution analysis of product list prices was performed using a pie chart. The analysis considered the top 10 list-price values and displayed their frequency distribution. This provided a visual understanding of the commonly occurring product price values in the dataset.

5.10 Customer Distribution by State

A bar chart was created to visualize the number of customers across different states. The state values were counted using Pandas and plotted using Matplotlib. This visualization helped identify the geographical concentration of customers in the dataset.

5.11 RFM Analysis

RFM analysis was performed to understand customer purchasing behavior using three important metrics: Recency, Frequency, and Monetary.

- Recency represents the number of days since the customer's most recent order.
- Frequency represents the number of orders associated with the customer.
- Monetary represents the total purchase value generated by the customer.

The orders and order-items datasets were merged using order_id, after which these three customer-level metrics were calculated.

5.12 Creating RFM Customer Data

The calculated Recency, Frequency, and Monetary values were combined into a new rfm_data DataFrame. The resulting dataset contained 570 customers with their corresponding RFM values. This dataset provided the foundation for customer value-based segmentation.

5.13 Customer Value Segmentation

Customers were segmented according to their total Monetary value. A custom segmentation function was created using predefined spending thresholds:

- **High Value:** Monetary value greater than 5,000
- **Medium Value:** Monetary value greater than 2,000 and up to 5,000
- **Low Value:** Monetary value of 2,000 or below

The resulting segment was added to the rfm_data DataFrame.

5.14 Exporting Customer Segmentation to SQL

The completed RFM segmentation results were exported back into the MySQL database using the Pandas to_sql() function. The results were stored in a table named customer_segments. This table can subsequently be connected to Power BI for customer segment analysis and visualization.

5.15 Python Analysis Summary

The Python stage completed the required data loading, exploratory analysis, visualization, RFM calculation, customer value segmentation, and export of segmentation results to SQL. The analysis produced a customer segmentation dataset containing Recency, Frequency, Monetary, and Segment fields, which forms the input for the next stage of the project—Power BI reporting

```
[5]: engine = create_engine("mysql+pymysql://root:12345@localhost:3306/sales")
[6]: file1 = pd.read_sql("SELECT * FROM orders; ", engine)
[7]: file1
```

[7]:

	order_id	customer_id	order_status	order_date	required_date	shipped_date	store_id	staff_id
0	1	259	4	1/1/2016	1/3/2016	1/3/2016	1	2
1	2	1212	4	1/1/2016	1/4/2016	1/3/2016	2	6
2	3	523	4	1/2/2016	1/5/2016	1/3/2016	2	7
3	4	175	4	1/3/2016	1/4/2016	1/5/2016	1	3
4	5	1324	4	1/3/2016	1/6/2016	1/6/2016	2	6
...	...	...	...	...	...	...	...	...
1610	1611	6	3	9/6/2018	9/6/2018	NaN	2	7
1611	1612	3	3	10/21/2018	10/21/2018	NaN	1	3
1612	1613	1	3	11/18/2018	11/18/2018	NaN	2	6
1613	1614	135	3	11/28/2018	11/28/2018	NaN	3	8
1614	1615	136	3	12/28/2018	12/28/2018	NaN	3	8

1615 rows × 8 columns

```
[9]: file2= pd.read_sql("SELECT * FROM order_items; ", engine)
[10]: file2
```

[10]:

	order_id	item_id	product_id	quantity	list_price	discount	product_name	total_price
0	1	1	20	1	599.99	0.20	Electra Townie Original 7D EQ - Women's - 2016	599.79
1	1	2	8	2	1799.99	0.07	Electra Townie Original 7D EQ - Women's - 2016	599.92
2	1	3	10	2	1549.00	0.05	Surly Wednesday Frameset - 2016	999.94
3	1	4	16	2	599.99	0.05	Ritchey Timberwolf Frameset - 2016	1499.88
4	1	5	4	1	2899.99	0.20	Surly Straggler - 2016	3097.95
...	...	...	...	...	...	...	...	...
1610	575	2	12	1	549.99	0.20	Electra Loft Go! 8i - 2018	2799.92
1611	576	1	9	1	2999.99	0.10	Electra Townie Commute 8D Ladies' - 2018	699.89
1612	577	1	3	2	999.99	0.20	Trek Domane SL 7 Women's - 2018	4999.92
1613	578	1	21	1	269.99	0.07	Surly Krampus - 2018	1498.93
1614	578	2	25	2	499.99	0.20	Trek Verve+ Lowstep - 2018	4599.78

1615 rows × 8 columns

```
11]: file3= pd.read_sql("SELECT * FROM customers; ", engine)
12]: file3
```

12]:

	customer_id	first_name	last_name	phone	email	street	city	state	zip_code
0	1	Debra	Burks	NA	debra.burks@yahoo.com	9273 Thorne Ave.	Orchard Park	NY	14127
1	2	Kasha	Todd	NA	kasha.todd@yahoo.com	910 Vine Street	Campbell	CA	95008
2	3	Tameka	Fisher	NA	tameka.fisher@aol.com	769C Honey Creek St.	Redondo Beach	CA	90278
3	4	Daryl	Spence	NA	daryl.spence@aol.com	988 Pearl Lane	Uniondale	NY	11553
4	5	Charolette	Rice	(916) 381-6003	charolette.rice@msn.com	107 River Dr.	Sacramento	CA	95820
...	...	...	...	...	...	...	...	...	...
1440	1441	Jamaal	Morrison	NA	jamaal.morrison@msn.com	796 SE. Nut Swamp St.	Staten Island	NY	10301
1441	1442	Cassie	Cline	NA	cassie.cline@gmail.com	947 Lafayette Drive	Brooklyn	NY	11201
1442	1443	Lezlie	Lamb	NA	lezlie.lamb@gmail.com	401 Brandywine Street	Central Islip	NY	11722
1443	1444	Ivette	Estes	NA	ivette.estes@gmail.com	88 N. Canterbury Ave.	Canandaigua	NY	14424
1444	1445	Ester	Acevedo	NA	ester.acevedo@gmail.com	671 Miles Court	San Lorenzo	CA	94580

1445 rows × 9 columns

```
[40]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1615 entries, 0 to 1614
Data columns (total 8 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   order_id      1615 non-null   int64
 1   item_id       1615 non-null   int64
 2   product_id    1615 non-null   int64
 3   quantity      1615 non-null   int64
 4   list_price    1615 non-null   float64
 5   discount      1615 non-null   float64
 6   product_name  1615 non-null   str
 7   total_price   1615 non-null   float64
dtypes: float64(3), int64(4), str(1)
memory usage: 101.1 KB
```

```
[41]:
```

```
[41]: df.describe()
```

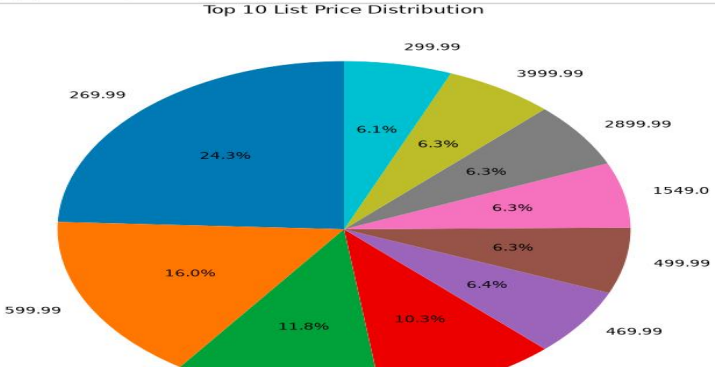
[41]:

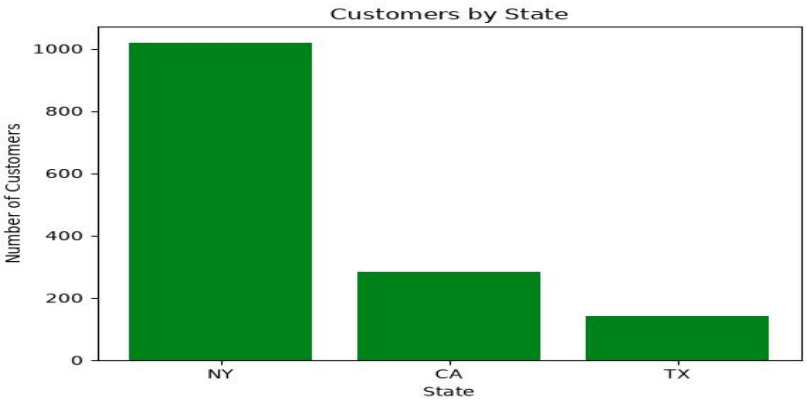
	order_id	item_id	product_id	quantity	list_price	discount	total_price
count	1615.000000	1615.000000	1615.000000	1615.000000	1615.000000	1615.000000	1615.000000
mean	287.592570	2.172136	13.889164	1.497833	1022.361189	0.104136	1743.317430
std	167.865457	1.140704	7.205393	0.500150	991.411428	0.057788	2117.282454
min	1.000000	1.000000	2.000000	1.000000	269.990000	0.050000	89.790000
25%	140.000000	1.000000	8.000000	1.000000	449.000000	0.050000	539.780000
50%	286.000000	2.000000	14.000000	1.000000	549.990000	0.070000	939.910000
75%	433.000000	3.000000	20.000000	2.000000	1434.995000	0.100000	1799.940000
max	578.000000	5.000000	26.000000	2.000000	3999.990000	0.200000	23999.880000

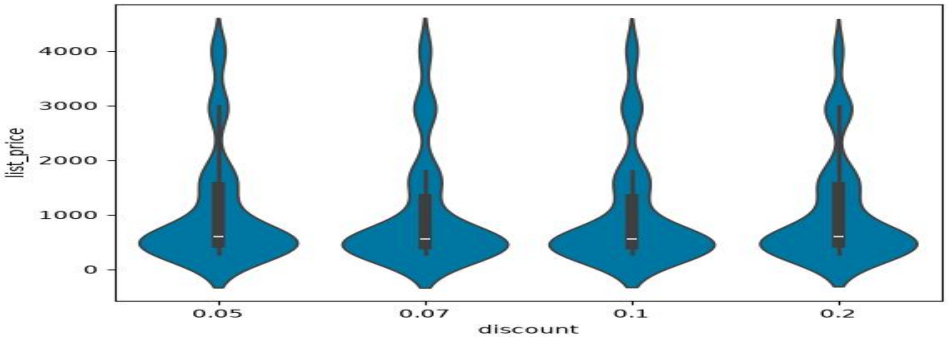
```
[42]: df.count()
```

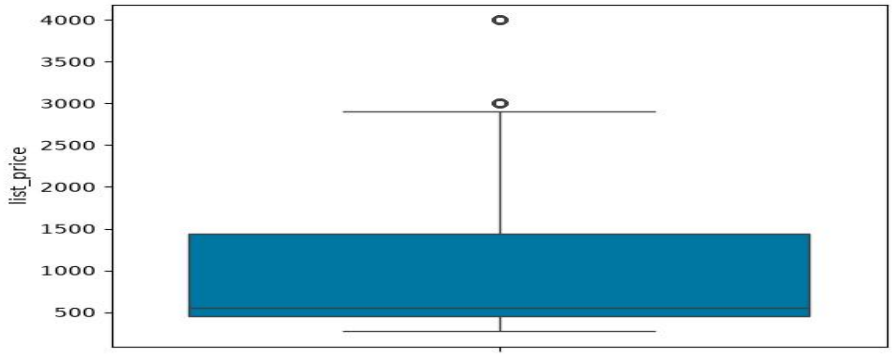
```
[42]: order_id        1615
      item_id         1615
      product_id      1615
      quantity        1615
      list_price      1615
      discount        1615
      product_name    1615
      total_price     1615
      dtype: int64
```


Monthly Orders Trend
Number of Orders
Month
220
200
180
160
140
120
100
2
4
6
8
10
12


Top 10 List Price Distribution
299.99
3999.99
269.99
2899.99
6.1%
6.3%
24.3%
6.3%
1549.0
6.3%
6.3%
499.99
16.0%
6.4%
599.99
11.8%
10.3%
469.99


Customers by State
Number of Customers
State
1000
800
600
400
200
0
NY
CA
TX


list_price
discount
4000
3000
2000
1000
0
0.05
0.07
0.1
0.2




```
[22]: print(rfm_data)

             Recency  Frequency  Monetary
customer_id
5               3542          6   8565.21
7               3808          2   8999.85
8               3578          1   2299.79
9               3824          3  12910.74
12              3805          3   2970.69
...              ...        ...       ...
1434            3844          2   1548.78
1435            3728          5  12876.43
1437            3630          1   1239.88
1440            3599          5   3245.46
1443            3551          5  13779.31

[570 rows x 3 columns]
```

6 – POWER BI: VISUALIZATION AND DASHBOARDING

6.1 Connecting Power BI to SQL

Power BI was connected to the MySQL database to import the required retail sales tables for analysis and visualization. The main tables used for dashboard development included orders, order_items, products, customers, stores, staffs, and customer_segments. This enabled the cleaned and analyzed data from the previous stages to be transformed into an interactive business intelligence dashboard.

6.2 Creating Relationships Between Tables

Relationships were created between the imported tables based on the primary and foreign keys defined in the SQL database. These relationships allowed data from customers, orders, products, stores, staffs, and customer segmentation to interact correctly across the dashboard visuals. This follows the project requirement to define relationships in Power BI Model View based on the ER diagram.

6.3 Dashboard Overview

An interactive Retail Sales Analytics Dashboard was developed in Power BI to provide an overall view of retail business performance. The dashboard uses KPI cards, charts, slicers, navigation buttons, and other interactive visuals to make the analysis easier to understand.

The dashboard contains dedicated pages for Customers, Products, Inventory, Staff & Sales, and Sales Analysis, along with an overview page for navigation and summary information.

6.4 Sales Overview Analysis

The Sales page was created to analyze overall sales performance and sales trends. A time-based sales visualization was used to identify changes in sales over time. A monthly sales trend was also included to understand variations in sales activity across different months.

The dashboard also provides sales-related summary information that helps management monitor overall business performance.

6.5 Sales Trend Analysis

A Sales Trend Over Time visualization was created to show how sales change over the selected period. This helps identify increasing or decreasing sales patterns and provides a clear view of overall sales performance.

A Monthly Sales Trend visualization was also included to support month-wise comparison of sales performance.

6.6 Product Sales Analysis

Product performance was analyzed using visualizations such as Sales by Product and Quantity Sold by Product. These visuals help identify products that contribute significantly to sales and products with higher sales quantities.

This analysis supports product performance evaluation and helps identify products that may require additional business attention.

6.7 Customer Purchase Analysis

The Customers page was developed to analyze customer purchasing behavior. A Customer Distribution by Segment visualization was included to compare customers across the available customer segments.

Customer analysis helps understand customer value and supports the identification of different customer groups based on their purchasing behavior.

6.8 Customer Segment Analysis

The customer segmentation results generated during the Python RFM analysis were imported into Power BI through the customer_segments table. The dashboard includes a Customer Segment field and a Revenue by Customer Segment visualization.

This allows the business to compare revenue contribution across different customer segments and understand which customer groups generate greater value.

6.9 Inventory Analysis

An Inventory page was created to monitor product stock levels. The dashboard includes Inventory Quantity by Category and Product Stock Details visuals.

The inventory analysis helps identify the quantity of products available across categories and provides a detailed view of product stock information.

6.10 Low Stock Monitoring

A low-stock analysis was included to support inventory monitoring. Products with stock levels below the defined threshold can be identified using the inventory information.

The project task specifies a low-stock threshold of less than 10 units, allowing businesses to identify products that may require replenishment.

6.11 Staff Performance Analysis

A dedicated Staff & Sales analysis page was created to evaluate employee sales performance. The dashboard includes Sales by Staff and Staff Sales Performance visuals.

This analysis allows management to compare revenue generated by individual staff members and identify differences in staff performance.

6.12 Regional Sales Analysis

A geographical analysis was included using a map visualization to show sales distribution by location. The project requirements specify using store location information such as state or ZIP code for geographical sales analysis.

The dashboard also includes Total Orders by City, which provides additional information about order activity across different locations.

6.13 Order Status Analysis

An Order Status visualization and slicer were included to analyze the distribution of orders across different status categories. This allows users to filter and examine order-related information based on order status.

6.14 Interactive Filters and Slicers

Interactive slicers were added to improve dashboard usability. The dashboard supports filtering based on important business dimensions such as Order Status, Product Category, Store, and Customer Segment.

These filters allow users to dynamically explore the dashboard and analyze specific portions of the retail data. The project requirements specifically include Order Status, Product Category, and Store slicers.

6.15 Dashboard Navigation

Navigation buttons and bookmark-based interactions were implemented to provide an organized dashboard experience. Users can move between different analytical pages and views without manually navigating through every report page.

This improves usability and allows the dashboard to function as an interactive reporting application rather than a collection of individual charts.

6.16 Consolidated Dashboard

The final dashboard combines the major retail analytics areas into a single interactive reporting solution. The project requirements specify a consolidated dashboard containing KPIs such as Total Revenue, Active Customers, Average Order Value, and Total Orders.

The dashboard brings together sales, customers, products, inventory, staff, order status, and customer segmentation information to provide a comprehensive view of retail business performance.

6.17 Power BI Dashboard Analysis Summary

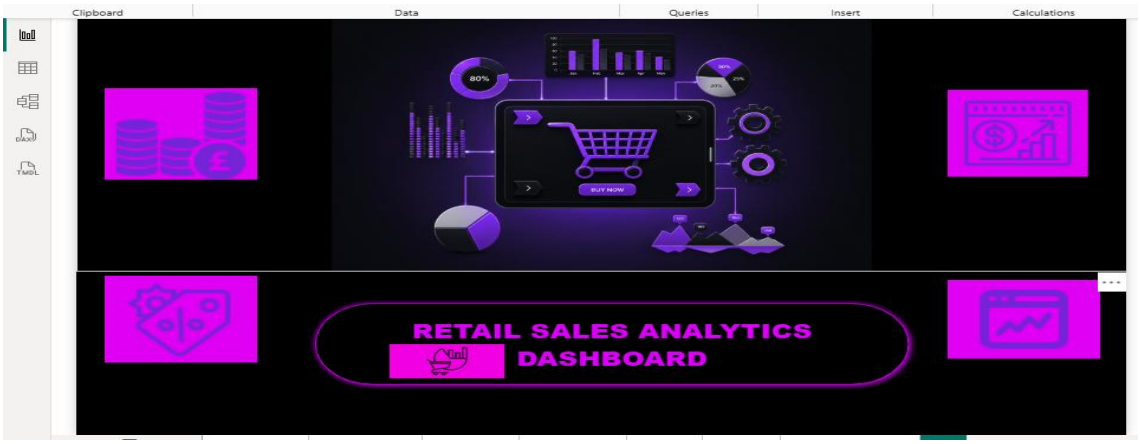
The Power BI stage transformed the SQL and Python outputs into an interactive business intelligence solution. The dashboard provides analysis of sales trends, product performance, customer segments, inventory, staff performance, order status, and regional activity.

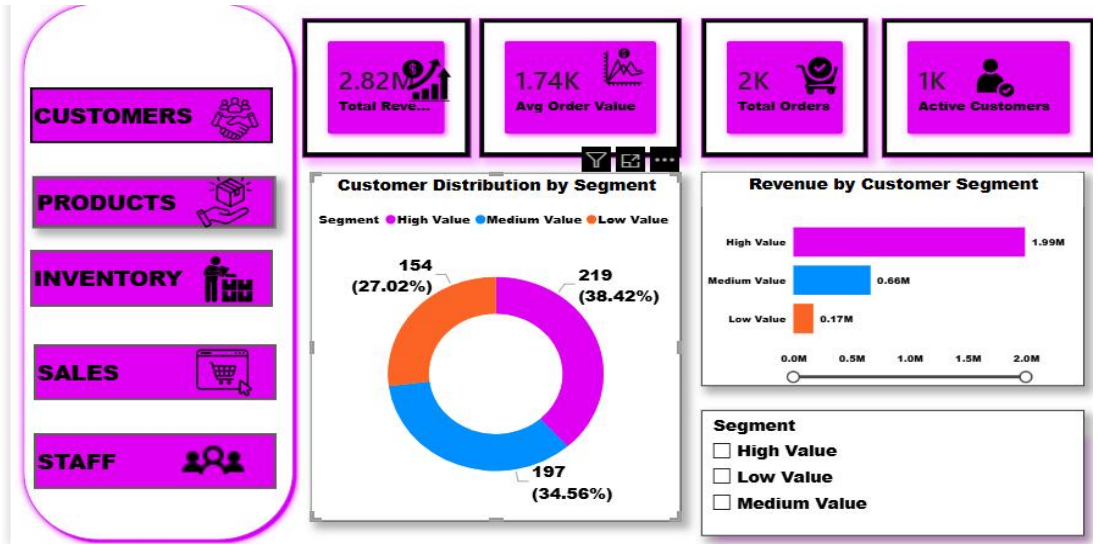
The dashboard follows the project objective of creating an interactive Power BI reporting solution and supports business users in exploring retail performance through visual analysis and interactive filters.

6.18 Key Business Insights

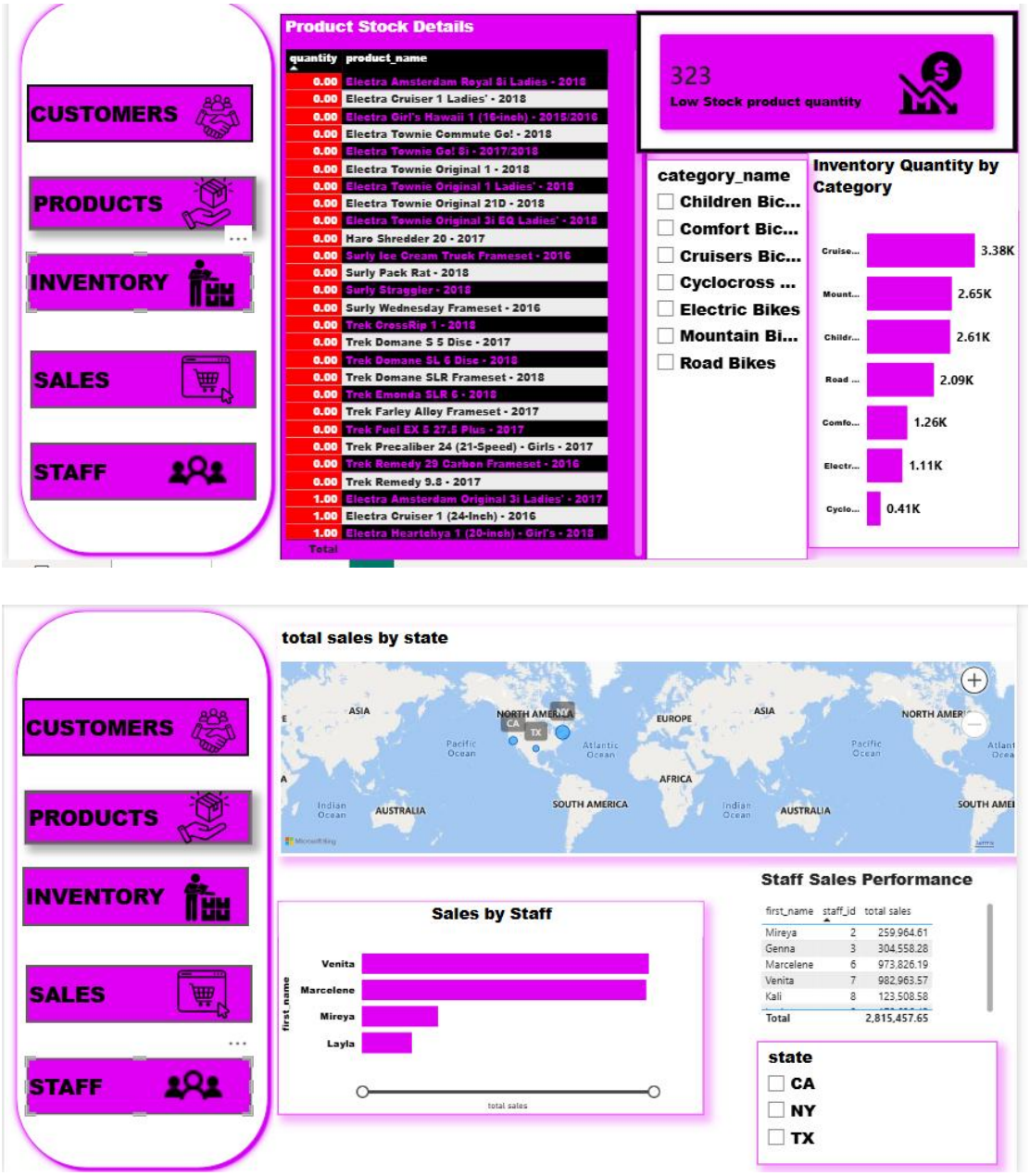
Based on the dashboard analysis, the following areas can be monitored:

- Sales trends can be compared over time to identify changes in business performance.
- Product sales and quantity can be compared to identify important products.
- Customer segments can be analyzed based on their revenue contribution.
- Inventory quantity can be monitored across product categories.
- Staff sales performance can be compared to understand employee contribution.
- Order activity can be analyzed by status and location.
- Interactive slicers allow users to investigate specific categories, stores, order statuses, and customer segments.


Clipboard
Data
Queries
Insert
Calculations
RETAIL SALES ANALYTICS
DASHBOARD


CUSTOMERS
PRODUCTS
INVENTORY
SALES
STAFF
2.82M
Total Reve...
1.74K
Avg Order Value
2K
Total Orders
1K
Active Customers
Customer Distribution by Segment
Segment High Value Medium Value Low Value
154
(27.02%)
219
(38.42%)
197
(34.56%)
Revenue by Customer Segment
High Value
1.99M
Medium Value
0.66M
Low Value
0.17M
0.0M
0.5M
1.0M
1.5M
2.0M
Segment
High Value
Low Value
Medium Value


CUSTOMERS
PRODUCTS
INVENTORY
SALES
STAFF
321
Total Products
2K
Total Quantity
Quantity Sold by Product
100%
Trek X-Caliber 8 - 20...
11
Trek Verve+ Lowstep...
4
Trek XM700+ - 2018
4
Trek XM700+ Lowst...
3
Trek X-Caliber 8 - 20...
1
9.1%
Sales by Product
product_name
Electra ...
0.15M
Surly Str...
0.13M
Trek Sla...
0.12M
Electra ...
0.12M
Pure Cyc...
0.10M
Electra ...
0.09M
Pure Cyc...
0.08M
total sales
category_name
Children Bicycles
Comfort Bicycles
Cruisers Bicycles
Cyclocross Bicycles



7 – RESULTS, BUSINESS INSIGHTS AND CONCLUSION

7.1 Results

The project successfully completed an end-to-end **Retail Sales Analytics** workflow using Excel, SQL, Python, and Power BI. The retail data was cleaned and prepared in Excel, stored and analyzed using MySQL, and further analyzed in Python using Exploratory Data Analysis and RFM-based customer segmentation. The customer segmentation results were exported to the SQL customer_segments table and incorporated into Power BI. Finally, an interactive dashboard was developed to analyze sales, customers, products, inventory, staff performance, order status, stores, and customer segments.

7.2 Business Insights

The analysis provides insights into sales performance, product performance, customer purchasing behavior, customer segmentation, inventory levels, staff performance, and store activity. RFM analysis identified customers based on Recency, Frequency, and Monetary values and classified them into High Value, Medium Value, and Low Value segments. Sales and product analysis helps identify important products and sales trends, while inventory analysis helps monitor low-stock products. Staff and store analysis provides additional information for evaluating business performance.

7.3 Conclusion

The Retail Sales Analytics Using Excel, SQL, Power BI and Python project successfully demonstrates a complete data analytics workflow. Excel was used for data cleaning and preparation, SQL for database management and querying, Python for exploratory analysis and RFM customer segmentation, and Power BI for interactive visualization and business reporting. The project transformed raw retail data into structured information and meaningful business insights that can support better sales monitoring, customer management, product analysis, inventory planning, and data-driven decision-making.

7.4 Final Project Summary

The final project follows the complete analytics pipeline:

Excel → SQL → Python → Power BI

The project demonstrates the practical application of data cleaning, database management, SQL querying, exploratory data analysis, RFM customer segmentation, data visualization, and business intelligence reporting. The final Power BI dashboard provides an interactive solution for analyzing key areas of retail business performance and supports management in making informed, data-driven decisions.